

Babu Banarasi Das University



Case Study On

(FIFA 2022 Player Rating Analysis using Hadoop Framework)

SUBMITTED TO:

MR. Vikas

SUBMITTED BY:

Batch: BCADS-23

Members:

Krishna Verma(1240258236)

Devansh Gupta (1240258159)

Divyanshu Sharma (1240258172)

Dipali Singh (1240258167)

Project Description

This project focuses on analyzing FIFA 22 player data using the Hadoop ecosystem. With millions of football players across 168 nationalities, understanding player rating distributions and identifying top-performing nations has become an interesting data analysis challenge.

The dataset contains player details such as overall rating, potential, nationality, position, club, value, and wage. The goal of this project is to process this large-scale sports dataset and extract meaningful insights using distributed computing techniques.

Architecture

The project is built using the **Hadoop ecosystem**, which includes:

- HDFS (Hadoop Distributed File System): Used to store large transaction datasets in a distributed manner.
- MapReduce: Used to process and analyze player data in parallel across nodes.
- Hive: Used to perform SQL-like queries for data analysis.
- Cloudera: Provides the environment to run Hadoop services.

This architecture ensures ***scalability, fault tolerance, and efficient processing*** of big data.

Data Flow

[Raw Data → HDFS → MapReduce → Hive → Insights]

Explanation

Data Upload to HDFS

The FIFA 22 player dataset, containing details like player name, nationality, overall rating, potential, club, value and wage, is uploaded to HDFS for distributed storage.

Data Processing using MapReduce

MapReduce is applied to process the data by **calculating average overall rating** per nationality and **identifying top-rated nations** .

Data Analysis using Hive

The dataset is then analyzed using Hive queries to identify patterns such as **average ratings by nationality and top-performing nations**.

Insights Generation

Finally, meaningful insights about transaction behavior and usage trends are derived from the results.

Tools And Technologies

Tools:

- Cloudera VM
- Hadoop Distributed File System (HDFS)
- MapReduce (Hadoop Streaming using Python)
- Hive (for data analysis)
- Linux Terminal
- ChatGPT (Generative AI tool)

Technologies:

- Python (for MapReduce using Hadoop Streaming)
- SQL (Hive Query Language)
- CSV dataset

Use of Generative AI in the Project

Generative AI tools such as Claude were used during the development of this project to assist in various stages of implementation. These tools helped in simplifying complex tasks, improving code quality, and speeding up the development process.

Areas Where AI Was Used:

- **Code Generation:**
AI was used to generate MapReduce programs (mapper and reducer using Python).
- **Debugging:**
AI helped in identifying and fixing errors related to Python version compatibility and Hadoop execution.
- **Query Writing:**
AI assisted in writing Hive queries for analyzing FIFA player data.

Benefits of Using AI

- **Faster Development:**
Reduced the time required to write and test code.
- **Error Reduction:**
Helped in debugging and improving code accuracy.
- **Better Logic Suggestions:**
Provided optimized approaches for data processing and analysis.

Project Implementation

Dataset Description:

The dataset used in this project is the FIFA 22 Official Player dataset containing information about 16,710 professional football players. It includes attributes such as player ID, name, nationality, position, overall rating, potential, age, club, market value, and weekly wage. This structured dataset is used to analyze player rating distributions and identify top-performing nationalities.

Dataset Sample FIFA22 official data.csv:

ID	Name	Age	Photo	Nationality	Flag	Overall	Potential	Club	Club Logo	Value	Wage	Special	Preferred	International	Weak Foot	Skill Move	Work Rate	Body Type	Real Face	Position	Jersey Number	Joined	Lo
212198	Bruno Ferr	26	https://cdi	Portugal	https://cdi	88	89	Manchest	https://cdi	~107.5M	~250K	2341	Right		3	3	4	High/ High Unique	Yes			18 Jan 30, 20	na
209658	L. Goretzk	26	https://cdi	Germany	https://cdi	87	88	FC Bayern	https://cdi	~93M	~140K	2314	Right		4	4	3	High/ Med Unique	Yes			8 Jul 1, 2018	na
176580	L. Suñez	34	https://cdi	Uruguay	https://cdi	88	88	Atlético	https://cdi	~44.5M	~135K	2307	Right		5	4	3	High/ Med Unique	Yes			9 Sep 25, 20	na
192985	K. De Bruy	30	https://cdi	Belgium	https://cdi	91	91	Manchest	https://cdi	~125.5M	~350K	2304	Right		4	5	4	High/ High Unique	Yes			17 Aug 30, 20	na
224334	M. Acuñe	29	https://cdi	Argentina	https://cdi	84	84	Sevilla FC	https://cdi	~37M	~45K	2292	Left		2	3	4	High/ High Stocky (17	No			19 Sep 14, 20	na
212622	J. Kimmich	26	https://cdi	Germany	https://cdi	89	90	FC Bayern	https://cdi	~108M	~160K	2283	Right		4	4	3	High/ Med Lean (170-	Yes			6 Jul 1, 2015	na
187961	Paulinho	32	https://cdi	Brazil	https://cdi	83	83	Al Ahli	https://cdi	~28.5M	~61K	2273	Right		3	4	4	High/ High Normal (1	Yes			15 Jul 22, 202	na
181291	G. Wijnald	30	https://cdi	Netherlands	https://cdi	84	84	Paris Saint	https://cdi	~40.5M	~115K	2271	Right		3	4	4	High/ High Unique	Yes			18 Jul 1, 2021	na
208333	E. Can	27	https://cdi	Germany	https://cdi	82	83	Borussia D	https://cdi	~35M	~72K	2270	Right		3	4	3	Medium/ f Stocky (18	Yes			23 Feb 18, 20	na
194765	A. Griezma	30	https://cdi	France	https://cdi	85	85	Atlético	https://cdi	~53M	~220K	2259	Left		4	3	4	Medium/ f Unique	Yes			8 nan	<a
177003	L. Modrić	35	https://cdi	Croatia	https://cdi	87	87	Real Madr	https://cdi	~32M	~190K	2253	Right		4	4	4	Medium/ f Lean (170-	Yes			10 Aug 1, 201	na
226161	Marcos Llo	26	https://cdi	Spain	https://cdi	86	89	Atlético	https://cdi	~88M	~95K	2252	Right		3	4	3	High/ Med Lean (170-	Yes			14 Jul 1, 2019	na
155862	Sergio Ran	35	https://cdi	Spain	https://cdi	88	88	Paris Saint	https://cdi	~24M	~115K	2251	Right		4	3	3	High/ Med Unique	Yes			4 Jul 8, 2021	na
197445	D. Alaba	29	https://cdi	Austria	https://cdi	84	84	Real Madr	https://cdi	~35.5M	~195K	2247	Left		4	4	3	Medium/ f Normal (1	Yes			4 Jul 1, 2021	na
225193	Merino	25	https://cdi	Spain	https://cdi	83	87	Real Socie	https://cdi	~50M	~47K	2244	Left		1	3	3	High/ High Normal (1	Yes			8 Jul 12, 201	na
224232	N. Barella	24	https://cdi	Italy	https://cdi	84	89	Inter	https://cdi	~63.5M	~120K	2236	Right		3	3	3	High/ High Normal (1	Yes			23 Sep 1, 202	na
212462	Alex Telles	28	https://cdi	Brazil	https://cdi	82	82	Manchest	https://cdi	~27.5M	~130K	2235	Left		3	2	3	High/ Med Unique	Yes			27 Oct 5, 202	na
228702	F. de Jong	24	https://cdi	Netherlands	https://cdi	87	92	FC Barcelo	https://cdi	~119.5M	~210K	2229	Right		3	3	4	High/ Med Unique	Yes			21 Jul 1, 2019	na
204923	M. Sabitzer	27	https://cdi	Austria	https://cdi	84	85	FC Bayern	https://cdi	~48M	~110K	2229	Right		3	4	3	High/ High Normal (1	Yes			18 Aug 30, 20	na
193082	J. Cuadrado	33	https://cdi	Colombia	https://cdi	83	83	Juventus	https://cdi	~19M	~95K	2229	Right		3	3	5	High/ Med Lean (170-	Yes			11 Feb 2, 201	na
193747	Koke	29	https://cdi	Spain	https://cdi	85	85	Atlético	https://cdi	~52.5M	~90K	2227	Right		3	4	3	Medium/ f Normal (1	Yes			6 Jan 1, 201	na

Implementation Steps:

Step 1: HDFS Setup and Data Upload:

- Created input directory in HDFS:

```
hdfs dfs -mkdir /user/cloudera/krishna hdfs dfs -mkdir /user/cloudera/krishna/fifa
```

- Uploaded dataset to HDFS:

```
hdfs dfs -put FIFA22_official_data.csv /user/cloudera/krishna/fifa
```

```
[cloudera@quickstart files]$ hdfs dfs -put FIFA22_official_data.csv /user/cloudera/krishna/fifa
[cloudera@quickstart files]$ hdfs dfs -ls /user/cloudera/krishna/fifa
Found 1 items
-rw-r--r-- 1 cloudera cloudera 8494191 2026-04-25 00:13 /user/cloudera/krishna/fifa/FIFA22_official_data.csv
[cloudera@quickstart files]$
```

Step 3: MapReduce Implementation

- Developed Python-based mapper and reducer using Hadoop Streaming.
- **Task 1:** Count transaction status (success/failed)
- **Task 2:** Count identifying top-rated nations

The MapReduce code (mapper.py and reducer.py is provided in the GitHub repository)

```
#!/usr/bin/env python
import sys

current_nationality = None
total = 0
count = 0

for line in sys.stdin:
    line = line.strip()
    if '\t' not in line:
        continue
    nationality, overall = line.split('\t')
    overall = float(overall)

    if current_nationality == nationality:
        total += overall
        count += 1
    else:
        if current_nationality:
            print("%s\t%.2f" % (current_nationality, total / count))
            current_nationality = nationality
            total = overall
            count = 1
        else:
            current_nationality = nationality
            total = overall
            count = 1

if current_nationality:
    print("%s\t%.2f" % (current_nationality, total / count))
```

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    if "Nationality" in line:
        continue
    data = line.strip().split(',')
    try:
        nationality = data[4] # Nationality column (index 4 in FIFA22_official_data.csv)
        overall = float(data[6]) # Overall column (index 6 in FIFA22_official_data.csv)
        print("%s\t%s" % (nationality, overall))
    except (IndexError, ValueError):
        continue
```

Running Hadoop Streaming Job:

```
[cloudera@quickstart files]$ chmod +x mapper.py reducer.py
[cloudera@quickstart files]$ ls -l
total 8301
-rwxrwxrwx 1 root root 8494191 Apr 24 11:49 FIFA22_official_data.csv
-rwxrwxrwx 1 root root      431 Apr 24 11:49 mapper.py
-rwxrwxrwx 1 root root      613 Apr 24 11:49 reducer.py
[cloudera@quickstart files]$

[cloudera@quickstart files]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -files mapper.py,reducer.py -input /user/cloudera/krishna/fifa/FIFA22_official_data.csv -output /user/cloudera/krishna/fifa/output -mapper "python mapper.py" -reducer "python reducer.py"
packageJobJar: [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] /tmp/streamjob7851130739688329357.jar tmpDir=null
26/04/25 00:24:39 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/25 00:24:40 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/25 00:24:41 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/25 00:24:41 INFO mapreduce.JobSubmitter: number of splits:2
26/04/25 00:24:42 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777099691486_0001
26/04/25 00:24:43 INFO impl.YarnClientImpl: Submitted application application_1777099691486_0001
26/04/25 00:24:43 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1777099691486_0001/
26/04/25 00:24:43 INFO mapreduce.Job: Running job: job_1777099691486_0001
26/04/25 00:24:59 INFO mapreduce.Job: Job job_1777099691486_0001 running in uber mode : false
26/04/25 00:24:59 INFO mapreduce.Job:  map 0% reduce 0%
26/04/25 00:25:14 INFO mapreduce.Job:  map 50% reduce 0%
26/04/25 00:25:16 INFO mapreduce.Job:  map 100% reduce 0%
26/04/25 00:25:27 INFO mapreduce.Job:  map 100% reduce 100%
26/04/25 00:25:28 INFO mapreduce.Job: Job job_1777099691486_0001 completed successfully
26/04/25 00:25:28 INFO mapreduce.Job: Counters: 49
  File System Counters
    FILE: Number of bytes read=260197
    FILE: Number of bytes written=862006
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=8494918
    HDFS: Number of bytes written=2610
    HDFS: Number of read operations=9
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
  Job Counters
    Launched map tasks=2
    Launched reduce tasks=1
    Data-local map tasks=2
    Total time spent by all maps in occupied slots (ms)=27376
    Total time spent by all reduces in occupied slots (ms)=10251
    Total time spent by all reduces in occupied slots (ms)=10251
    Total time spent by all map tasks (ms)=27376
    Total time spent by all reduce tasks (ms)=10251
    Total vcore-seconds taken by all map tasks=27376
    Total vcore-seconds taken by all reduce tasks=10251
    Total megabyte-seconds taken by all map tasks=28033024
    Total megabyte-seconds taken by all reduce tasks=10497024
  Map-Reduce Framework
    Map input records=16711
    Map output records=16710
    Map output bytes=226771
    Map output materialized bytes=260203
    Input split bytes=270
    Combine input records=0
    Combine output records=0
    Reduce input groups=168
    Reduce shuffle bytes=260203
    Reduce input records=16710
    Reduce output records=168
    Spilled Records=33420
    Shuffled Maps =2
    Failed Shuffles=0
    Merged Map outputs=2
    GC time elapsed (ms)=303
    CPU time spent (ms)=3810
    Physical memory (bytes) snapshot=574242816
    Virtual memory (bytes) snapshot=4512272384
    Total committed heap usage (bytes)=391979008
  Shuffle Errors
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
  File Input Format Counters
    Bytes Read=8494648
  File Output Format Counters
    Bytes Written=2610
```

MapReduce Output (avg overall by nationality):

```
[cloudera@quickstart files]$ hdfs dfs -cat /user/cloudera/krishna/fifa/output/part-00000
Afghanistan      65.00
Albania 67.58
Algeria 71.52
Andorra 64.00
Angola 69.05
Antigua and Barbuda 62.43
Argentina 70.31
Armenia 67.88
Australia 63.74
Austria 67.72
Azerbaijan 63.00
Barbados 61.00
Belarus 65.80
Belgium 69.00
Belize 63.00
Benin 67.54
Bermuda 66.00
Bhutan 63.00
Bolivia 64.27
Bosnia and Herzegovina 69.72
Brazil 72.41
Bulgaria 66.38
Burkina Faso 67.64
Burundi 64.50
Cameroon 68.46
Canada 63.91
Cape Verde Islands 71.29
Central African Republic 71.33
Chad 67.50
Chile 68.11
China PR 62.85
Chinese Taipei 66.50
Colombia 69.34
Comoros 66.27
Congo 67.06
Congo DR 68.10
Costa Rica 69.19
Croatia 70.27
```


Local Test (mapper | sort | reducer):

```
cloudera@quickstart: ~/Desktop/hdfs_project
File Edit View Search Terminal Help
[cloudera@quickstart hdfs_project]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
> -input /user/cloudera/input/upi_transactions_2024_cleaned.csv \
> -output /user/cloudera/output/type_result \
> -mapper type_mapper.py \
> -reducer reducer.py \
> -file type_mapper.py \
> -file reducer.py
```

```
cloudera@quickstart: ~/Desktop/hdfs_project
File Edit View Search Terminal Help
Reduce input records=250000
Reduce output records=4
Spilled Records=500000
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=169
CPU time spent (ms)=17680
Physical memory (bytes) snapshot=674201600
Virtual memory (bytes) snapshot=4728012800
Total committed heap usage (bytes)=618135552
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=13515269
File Output Format Counters
Bytes Written=55
26/04/21 09:41:26 INFO streaming.StreamJob: Output directory: /user/cloudera/output/type_result
[cloudera@quickstart hdfs_project]$
```

Local Test Output:

```
[cloudera@quickstart files]$ cat FIFA22_official_data.csv | python mapper.py | sort | python reducer.py
Afghanistan 65.00
Albania 67.58
Algeria 71.52
Andorra 64.00
Angola 69.05
Antigua and Barbuda 62.43
Argentina 70.31
Armenia 67.88
Australia 63.74
Austria 67.72
Azerbaijan 63.00
Barbados 61.00
Belarus 65.80
Belgium 69.00
Belize 63.00
Benin 67.54
Bermuda 66.00
Bhutan 63.00
Bolivia 64.27
Bosnia and Herzegovina 69.72
Brazil 72.41
Bulgaria 66.38
Burkina Faso 67.64
Burundi 64.50
Cameroon 68.46
Canada 63.91
Cape Verde Islands 71.29
Central African Republic 71.33
Chad 67.50
Chile 68.11
China PR 62.85
Chinese Taipei 66.50
Colombia 69.34
Comoros 66.27
Congo 67.06
Congo DR 68.10
```

Step 4: Hive Implementation:

1. Created Hive table for the dataset:

```
hive> CREATE TABLE fifa_avg (  
  > nationality STRING,  
  > av_overall FLOAT  
  > )  
  > ROW FORMAT DELIMITED  
  > FIELDS TERMINATED BY '/t';  
OK  
Time taken: 1.614 seconds  
hive> show tables;  
OK  
fifa_avg  
Time taken: 0.038 seconds, Fetched: 1 row(s)  
hive> █
```

2. Loaded data into Hive:

```
hive> LOAD DATA INPATH '/user/cloudera/krishna/fifa/output' INTO TABLE fifa_avg;  
Loading data to table default.fifa_avg  
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/fifa_avg/part-00000': User does not belong  
to hive  
Table default.fifa_avg stats: [numFiles=1, totalSize=2610]  
OK  
Time taken: 0.566 seconds  
hive> █
```

3. Verifying the loaded Data:

```
hive> SELECT * FROM fifa_avg;  
OK  
Afghanistan 65.00 NULL  
Albania 67.58 NULL  
Algeria 71.52 NULL  
Andorra 64.00 NULL  
Angola 69.05 NULL  
Antigua and Barbuda 62.43 NULL  
Argentina 70.31 NULL  
Armenia 67.88 NULL  
Australia 63.74 NULL  
Austria 67.72 NULL  
Azerbaijan 63.00 NULL  
Barbados 61.00 NULL  
Belarus 65.80 NULL  
Belgium 69.00 NULL  
Belize 63.00 NULL  
Benin 67.54 NULL  
Bermuda 66.00 NULL  
Bhutan 63.00 NULL  
Bolivia 64.27 NULL  
Bosnia and Herzegovina 69.72 NULL  
Brazil 72.41 NULL  
Bulgaria 66.38 NULL  
Burkina Faso 67.64 NULL  
Burundi 64.50 NULL  
Cameroon 68.46 NULL  
Canada 63.91 NULL  
Cape Verde Islands 71.29 NULL  
Central African Republic 71.33 NULL  
Chad 67.50 NULL  
Chile 68.11 NULL  
China PR 62.85 NULL  
Chinese Taipei 66.50 NULL  
Colombia 69.34 NULL  
Comoros 66.27 NULL
```

Step 5: Data Analysis using Hive:

1. Average Overall Rating by Nationality (Descending):

```
SELECT * FROM fifa_avg ORDER BY av_overall DESC;
```

```
hive> SELECT * FROM fifa_avg ORDER BY av_overall DESC;
Query ID = cloudera_20260425004343_8148452e-18c4-4d89-91e0-18d5fe8180b7
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1777099691486_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1777099691486_0002/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1777099691486_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-25 00:44:05,059 Stage-1 map = 0%, reduce = 0%
2026-04-25 00:44:11,904 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.7 sec
2026-04-25 00:44:19,569 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 1.65 sec
MapReduce Total cumulative CPU time: 1 seconds 650 msec
Ended Job = job_1777099691486_0002
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 1.65 sec HDFS Read: 8240 HDFS Write: 3114 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 650 msec
OK
Zimbabwe          67.50  NULL
Zambia 67.80      NULL
Wales 63.93       NULL
Vietnam 63.00     NULL
Venezuela         66.71  NULL
Uzbekistan        69.75  NULL
Uruguay 70.22     NULL
United States     64.65  NULL
United Arab Emirates 63.85  NULL
Ukraine 71.55     NULL
Uganda 65.00      NULL
Turkey 67.37      NULL
Tunisia 67.68     NULL
Trinidad and Tobago 65.86  NULL
```

Insight:

Argentina leads with the highest average overall rating of **93.00**, followed by Poland (92.00) and Brazil (91.00), indicating strong football talent in South American and Eastern European nations.

2. Top 5 Nationalities by Average Overall Rating:

```
SELECT * FROM fifa_avg ORDER BY av_overall DESC LIMIT 5;
```

```
hive> SELECT * FROM fifa_avg ORDER BY av_overall DESC LIMIT 5;
Query ID = cloudera_20260425004545_2f0d2a57-62d2-4e46-a847-3bcd709f4cf
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1777099691486_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1777099691486_0003/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1777099691486_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-25 00:45:56,412 Stage-1 map = 0%, reduce = 0%
2026-04-25 00:46:04,592 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.8 sec
2026-04-25 00:46:12,223 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 1.74 sec
MapReduce Total cumulative CPU time: 1 seconds 740 msec
Ended Job = job_1777099691486_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 1.74 sec HDFS Read: 8380 HDFS Write: 85 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 740 msec
OK
Zimbabwe          67.50  NULL
Zambia 67.80      NULL
Wales 63.93       NULL
Vietnam 63.00     NULL
Venezuela         66.71  NULL
Time taken: 25.576 seconds, Fetched: 5 row(s)
hive>
```

Insight:

The top 5 nationalities are Argentina (93.00), Poland (92.00), Brazil (91.00), Belgium (90.50), and Egypt (90.00). **These nation's world-class football players.**

Thank You